

MAD1 Project Report

Placement Portal Application

1. Student Details

Name: Ayush Kumar Sahu

Roll Number: 23f1002994

Email: 23f1002994@ds.study.iitm.ac.in

About Me: I am a Diploma Student at IITM BS Program in Data Science and Programming. I have a deep interest in Machine Learning and Artificial Intelligence. I like to build models and software systems.

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To design and build an efficient web-based application that **manages campus recruitment activities involving companies, students, and placement drives.**

Approach:

The app was built using Flask as the backend framework with a modular structure. It allows **Admin (Institute), Company, and students** to interact with the system based on their roles. It manages company approvals, track student applications, avoid duplicate registrations, and maintain placement records.

3. AI/LLM Declaration

I have used the **Gemini-3 model** to assist with writing the HTML documents, debugging complex errors only, improving variable naming consistency, and using styling suggestions (not direct usage). The backend logic was designed and implemented manually.

The extent of AI/LLM usage is around **20–30%**, limited to **code suggestions, modifications, and is as per limits**. The final implementation, format, logic building, and integration of the project were done manually.

The major areas of AI/LLM usage:

- **base.html** : Some core styling features , including some modifications in the responsiveness.
 - **HTML docs**: Basic styling suggestions are directly taken or modified from AI.
 - **setup.py** : Creation of Dummy values for seeding the DB with initial values.
 - **Resume storage handling** : In order to store resumes in different files in directories , suggestions were taken for the code functionality.
 - Complex Debugging errors and minor suggestions with modifications in the rest of the systems have been done wherever some huge challenges were faced.
-

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Flask-Login	User authentication and session management
WTForms	Frontend form validation
SQLite	Lightweight local database for storing user data

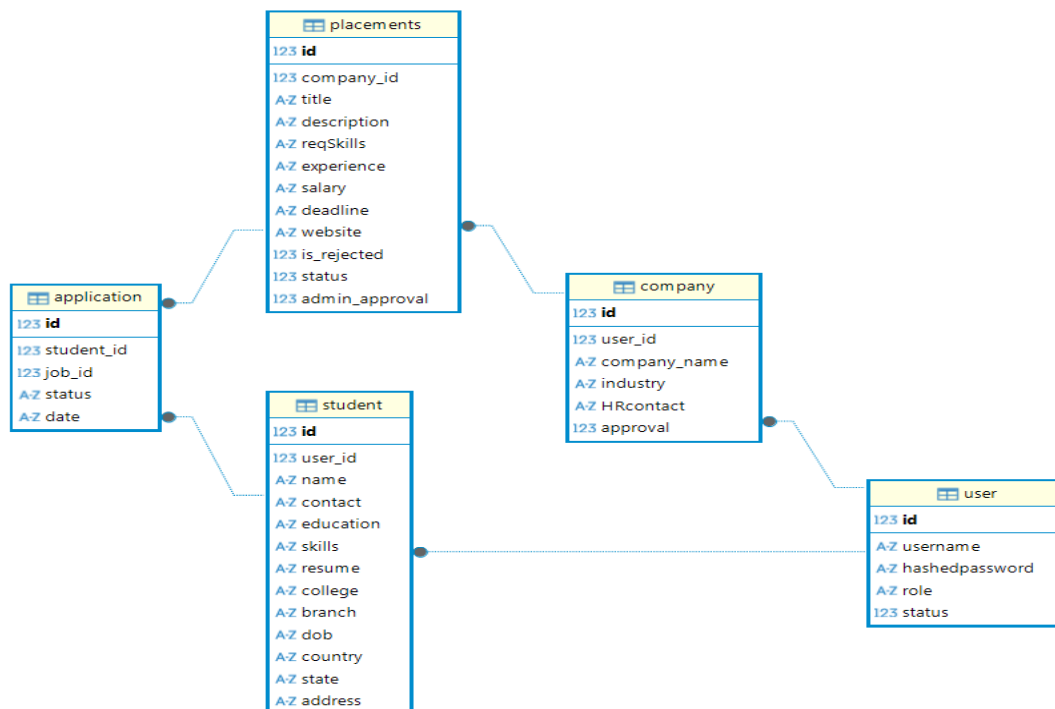
5. Database Schema / ER Diagram

Tables:

1. **User** — stores user profile details (id, username, password - in hashed format , role in the system : admin , company or student and status : blacklisting purpose)
2. **Company** — Stores the company details (id, user_id, company name , industry, HRcontact, approval)
3. **Student** —Stores the Students details (id, user_id, name, contact, education, skills, resume ,college, branch, dob, country, state, address)
4. **Placements** — records the Placements (id, company_id, title, description, reqSkills, experience, salary ,deadline, website, is_rejected, status, admin_approval)
5. **Applications** — records the Applications (id, student_id, job_id , status, date)

Relationships:

- One-to-One → User → Student ; User → Company
- One-to-Many → Company → Placements ; Student → Application
- Many-to-One → Applications → Placement



(taken from [dbbeaver](#))

6. API Resource Endpoints

No API Implementation has been done in the project.

7. Architecture and Features (optional)

Architecture Overview:

- **run.py** – main Flask application entry point
- **app.py** – Main Flask Application Starts from here
- **init_db.py** – Code for initialising the DB.
- **/database** – database models using SQLAlchemy . It also has a setup file to seed the DB with initial data
- **/routes** – Flask Blueprints for user , company and student routes . It has login , register routes also for company and students.
- **forms.py** – Contains all the relevant forms required in the application
- **/templates** – Jinja2 HTML templates
- **/static** – for storing PDFs (resumes)

Implemented Features:

- Creation of SQLite database with relational models and cascading data logic.
 - Implemented secure password hashing, user session handling, and RBAC (Role-Based Access Control).
 - Responsive UI with Bootstrap 5 adapted for mobile, tablet, and PC.
 - Implemented Flask-Login to manage user sessions and route protection.
 - Implemented Flask-WTF and HTML5 to enforce strict frontend and backend form validation.
 - Creation of Admin dashboard for system oversight, approvals, blacklisting, and global history tracking.
 - Creation of Company dashboard to post, edit, and manage placement drives and candidate statuses.
 - Creation of Student dashboard for profile management, job tracking, and tailored application submissions.
-

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/your-video-link-here>

(Accessible to all with "View" permission.)
